

DBT TRAINING

● Working Effectively · Module 12 · 90 min

CI/CD and Slim CI

How broken models get stopped before they reach production — and why it matters how fast CI runs.

It's 3am. A Dashboard is Broken.

Slack · #data-alerts · 03:14

Pipeline failed. mart_revenue is empty. Power BI dashboard showing zeros for all of March.

GitHub · main · 2h ago

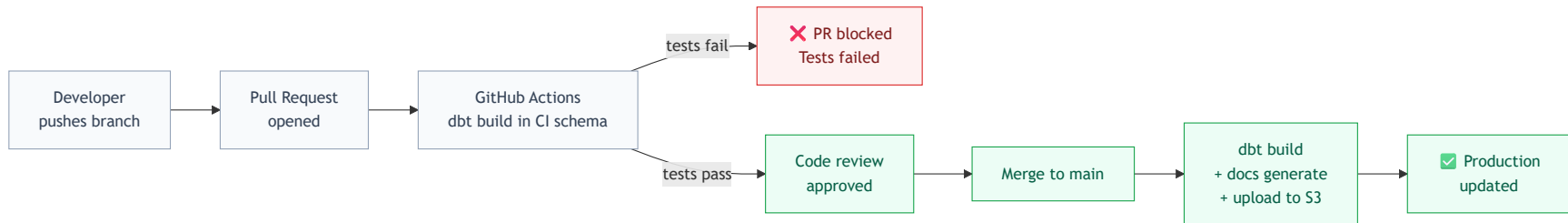
Merged: "fix deal stage mapping" — 3 files changed

What went wrong: A column rename broke a `not_null` test downstream. No CI. The code merged and ran against production.

- ✗ No CI — broken SQL reached production
- ✗ Tests existed but never ran automatically
- ✗ Someone had to find this at 3am

With CI: The PR is blocked. The merge never happens. Production never sees the broken model.

What CI/CD Means in dbt



CI (Continuous Integration)

Runs on every PR push. Builds models.
Runs tests. Blocks merge on failure.

CD (Continuous Delivery)

Runs on merge to main. Full build
against PROD. Publishes docs to S3.

The guarantee

If CI passes, every test you wrote
passed against real data. Broken
models cannot merge.

PR Workflow: What Runs When

`on: pull_request` every PR push

- ▶ Download prod `manifest.json` from S3
- ▶ `dbt deps`
- ▶ `dbt build --select state:modified+ --defer --state ./prod-artifacts --target ci`
- ▶ Drop CI schema (cleanup)

Writes to `DBT_CI_<run-id>` — a throwaway schema. Never touches PROD.

Parallel PR safety: Unique schema per run ensures parallel PRs don't corrupt each other's test results.

`on: push to main` after merge

- ▶ `dbt deps`
- ▶ `dbt build --target prod` (full project)
- ▶ `dbt docs generate`
- ▶ Upload `manifest.json` → S3 (for next Slim CI)
- ▶ Sync docs site → S3

Writes to **PUBLIC** — the real production schema.

The Cost Problem: Running Everything on Every PR

Without Slim CI

```
Project: 18 models
Average build time: 3 min/model

-----

Full dbt build: 18 × 3 = 54 min
PRs per week: 10

-----

CI compute/week: 540 min
CI compute/month: ~2,160 min
```

Most of that compute rebuilds models that did not change.

With Slim CI (typical PR: 1 model changed)

```
Modified: fct_deal
Downstream: mart_revenue
            mart_pipeline_summary

-----

Selected: 3 models × 3 min = 9 min
PRs/week: 10

-----

CI compute/week: 90 min
Savings: 450 min/week (83%)
```

Same test coverage. Fraction of the cost.

As the project grows, this gap widens. 50 models × 3 min = 150 min per PR. Slim CI still builds 3–5 models per typical PR.

Slim CI: `manifest.json` + `state:modified+`

What is `manifest.json` ?

Produced by every `dbt build`. Lives in `./target/`.

Contains: the full model graph + a **compiled SQL hash** for each model.

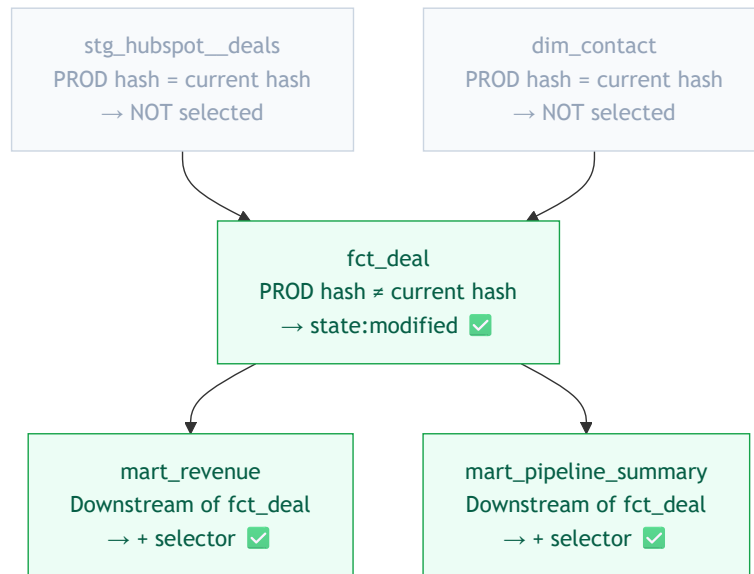
Uploaded to S3 after each production run.

Downloaded by CI before it runs.

What does `state:modified+` do?

Part	Selects
<code>state:modified</code>	Models whose SQL hash $\neq$ production hash
<code>+</code> (trailing)	All downstream dependents

Example: you changed `fct_deal.sql`



3 of 18 models selected. 15 skipped.

How `--defer` Works

GitHub Actions Walkthrough

```
# .github/workflows/dbt_ci.yml
name: dbt CI/CD

on:
  pull_request:
    branches: [main]
  push:
    branches: [main]

jobs:
  slim_ci:
    runs-on: ubuntu-latest
    if: github.event_name == 'pull_request'
    env:
      DBT_TARGET: ci
      SNOWFLAKE_ACCOUNT: ${ secrets.SNOWFLAKE_ACCOUNT }
      SNOWFLAKE_USER: ${ secrets.SNOWFLAKE_USER }
      SNOWFLAKE_PASSWORD: ${ secrets.SNOWFLAKE_PASSWORD }
      SNOWFLAKE_DATABASE: ${ secrets.SNOWFLAKE_DATABASE }
      SNOWFLAKE_WAREHOUSE: ${ secrets.SNOWFLAKE_WAREHOUSE }
      SNOWFLAKE_ROLE: ${ secrets.SNOWFLAKE_ROLE }
      DBT_SCHEMA: DBT_CI-${ github.run_id }

    steps:
```


Environment Variables and Targets

profiles.yml (committed — no secrets)

```
analytics:
  target: dev
  outputs:
    dev:
      type: snowflake
      account: "{{ env_var('SNOWFLAKE_ACCOUNT') }}"
      user:     "{{ env_var('SNOWFLAKE_USER') }}"
      password: "{{ env_var('SNOWFLAKE_PASSWORD') }}"
      schema:  "TESTING__{{ env_var('DBT_DEVELOPER_N

ci:
  type: snowflake
  account: "{{ env_var('SNOWFLAKE_ACCOUNT') }}"
  user:     "{{ env_var('SNOWFLAKE_USER') }}"
  password: "{{ env_var('SNOWFLAKE_PASSWORD') }}"
  schema:  "{{ env_var('DBT_SCHEMA', 'DBT_CI') }}"

prod:
  type: snowflake
  account: "{{ env_var('SNOWFLAKE_ACCOUNT') }}"
  user:     "{{ env_var('SNOWFLAKE_USER') }}"
```

Three targets, three schemas

Target	Used by	Schema
dev	Developer locally	TESTING__yourname
ci	GitHub Actions (PR)	DBT_CI_<run-id>
prod	GitHub Actions (merge)	PUBLIC

Rule: `profiles.yml` is committed. Never put a password in it. All credentials go through `env_var()`. Local devs set these in a `.env` file (git-ignored). CI reads from GitHub secrets.

Key point: CI uses target `ci`. The `--target ci` flag in the workflow tells dbt which output block to use. It never

Documentation Quality Gate

A PR is blocked if any of these are missing:

- ✗ A Silver or Gold model has no `description` in `schema.yml`
- ✗ The SQL file has no `-- grain:` comment at the top
- ✗ A `_key` column (surrogate or FK) has no `description`

How it is enforced

The reviewer runs the `dbt-sql-reviewer` skill before approving. Every failed check becomes a review comment. The PR cannot be approved until all checks pass.

A model that passes the gate

```
-- grain: one row per deal per day
-- Each row represents the state of a HubSpot deal
-- as recorded at the end of the calendar day.
```

```
{{ config(
    materialized = 'incremental',
    unique_key   = 'fct_deal_key',
    on_schema_change = 'sync_all_columns'
) }}
```

```
models:
  - name: fct_deal
    description: >
      Daily snapshot of HubSpot deal state. One row p
      per calendar day. Source of truth for pipeline
    columns:
      - name: fct_deal_key
        description: "Surrogate key – hash of deal_id"
      - name: deal_id
        description: "HubSpot deal ID"
```

Exercise — Fix the Broken CI Config (25 min)

Scenario: A colleague wrote the CI workflow below. It looks mostly right but has **three bugs**. Identify each one, explain why it is a problem, and write the fix. Then complete Tasks 2 and 3.

```
jobs:
  slim_ci:
    runs-on: ubuntu-latest
    env:
      DBT_TARGET: ci
      SNOWFLAKE_ACCOUNT: ${ secrets.SNOWFLAKE_ACCOUNT }}
      SNOWFLAKE_USER:    ${ secrets.SNOWFLAKE_USER }}
      SNOWFLAKE_PASSWORD: "Snowflake_Prod_2024_v3" # ← ?
      SNOWFLAKE_DATABASE: ${ secrets.SNOWFLAKE_DATABASE }}
      DBT_SCHEMA:          DBT_CI_${ github.run_id }}
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '3.11' }
      - run: pip install dbt-snowflake==1.8.4
      - name: Download production manifest
        run: |
          mkdir -p ./prod-artifacts
          aws s3 cp s3://bloomwell-dbt-artifacts/prod/manifest.json ./prod-artifacts/manifest.json
      - run: dbt deps
```

Exercise — Solutions

Bug 1 • Security

❌ **Hardcoded password in YAML**

```
# Wrong
SNOWFLAKE_PASSWORD: "Snowflake_Pr

# Fixed
SNOWFLAKE_PASSWORD: ${ secrets.S
```

YAML is committed to the repo. Anyone with read access sees the password. GitHub also flags this as an exposed secret.

Bug 2 • Tests not running

❌ `dbt run` instead of `dbt build`

```
# Wrong
dbt run --select state:modified+

# Fixed
dbt build --select state:modified
```

`dbt run` builds models only. Tests are skipped. CI can pass while every `not_null` test in the project fails silently.

Bug 3 • Wrong state path

❌ `--state ./target` (**local, empty**)

```
# Wrong
--state ./target

# Fixed
--state ./prod-artifacts
```

`./target` is the current run's output — empty at the start. The prod manifest was downloaded to `./prod-artifacts`. Wrong path = no baseline = Slim CI breaks.

Task 2 answer: `dbt build --select state:modified+ --defer --state ./prod-artifacts --target ci`

CI/CD pipeline

Broken models are stopped at the PR gate — never touch production.

Slim CI

```
state:modified+ + --  
defer cuts CI cost by 80%+  
on typical PRs.
```

Documentation gate

Pre-merge is the only moment that creates leverage. Use it.

● Tier 2 Complete

You've finished Working Effectively. You can build, test, document, schedule, and protect production dbt projects.

TIER 3 — ADVANCED PATTERNS

Module 13 — Advanced Testing Patterns

Module 14 — Incremental Strategies

Module 15 — SCD2 deep dive

Module 16 — Governance & Contracts

PREP FOR MODULE 13 — ADVANCED TESTING PATTERNS

Q1: Difference between not_null and relationships test?

Q2: Where do you define a reusable generic test?

Q3: What does severity: warn do vs. default error?